

Programování

- nejedná se o disciplínu „psaní na klávesnici“
- když jednou něco vytvoříte, znovu to použijte
- využívejte příklady z webu/prezentace a přizpůsobte si je
- programování se nedá naučit teoreticky, bez procvičení
na konkrétních příkladech
- důležité je umět přečíst hotový kód

Náplň cvičení

- deklarace proměnných, konstant
- komentáře
- výpis na monitor (výstup)
- čtení z klávesnice (vstup)

Základní „kostra“ programu

direktivy pro preprocesor

```
int main()
{
    ... vlastní program
    return 0;
}
```

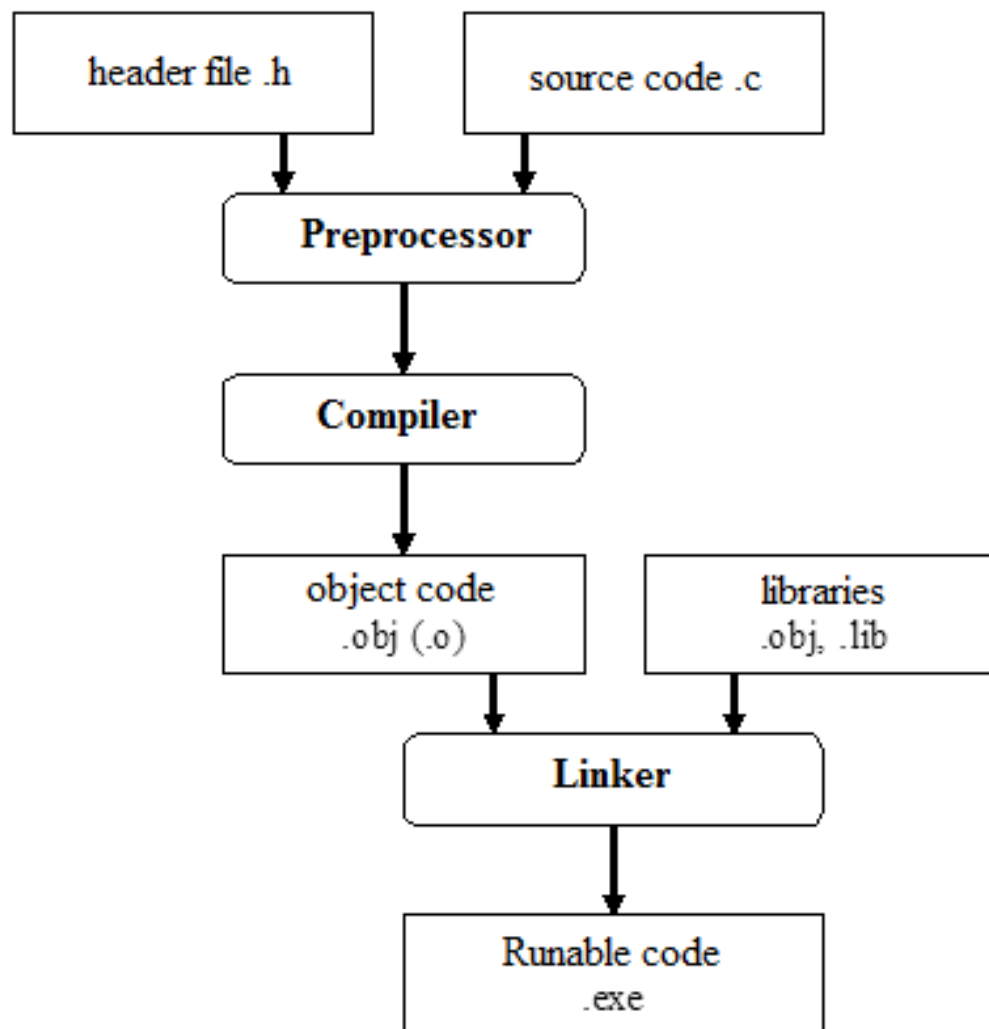
Doporučení/rady při programování

1. Každý příkaz musí být ukončen středníkem.
2. Čtěte chybové hlášky překladače – je tam uvedeno i číslo řádku, ve kterém překladač narazil na problém. Většinou se chyba v daném řádku nachází, ale problém může být i o řádek výše.
3. Většinou bývá chyba (překlep) v názvu funkce/příkazu, chybí nebo přebývá nějaké to písmenko.
4. Průběžně si kontrolujte překladem (samotný překlad - compile), že nemáte ve zdrojovém kódu (syntaktickou) chybu. Nedoporučuji nechávat překlad až na chvíli, kdy si myslíte, že máte vše naprogramováno. Při souběhu více chyb a absenci zkušeností se v tom ztratíte. (postupujte „per partes“)

Programovací jazyk C

- ROZLIŠUJE velikost písma (je tzv. „Case Sensitive“)
 - `promA`, `Proma`, `PROMa` jsou 3 různé proměnné

Překlad v jazyce C



Deklarace proměnných

datový_typ *název_proměnné*;

```
int a, b=20, c;
```

```
float x=5.87;
```

```
char znak='A';
```

Skupina	Typ
Znakový typ	char
Celočíselné typy	short
	int
	long
Typy v pohyblivé řádové čárce	float
	double
	long double

Po deklaraci proměnných „a“ a „c“ nemusí být jasný jejich obsah (hodnota)!!! (záleží na kompilátoru)

Lokální x Globální proměnná

```
#include <stdio.h>
```

```
double globalniX = 3.1415927;
```

```
int main()
```

```
{
```

```
    double lokalniX;
```

```
    ...
```

```
}
```


Práce s proměnnou

```
int a, b=20, c;
```

```
c=10;
```

```
a=b+c;
```

```
c=c+10;
```

Výstup na obrazovku

`printf(.....);` //nutná direktiva `#include <stdio.h>`

formátovací řetězec

`printf("Uz umim výpis!!!");`

specifikátor	význam
%d	celé číslo
%f	float
%c	char
%f (%lf)	double
%s	řetězec

`printf("Hodnota promenne: %d",c);`

Častá chyba

- Špatně zvolený specifikátor

```
int a,b,c;
```

```
float f;
```

```
...
```

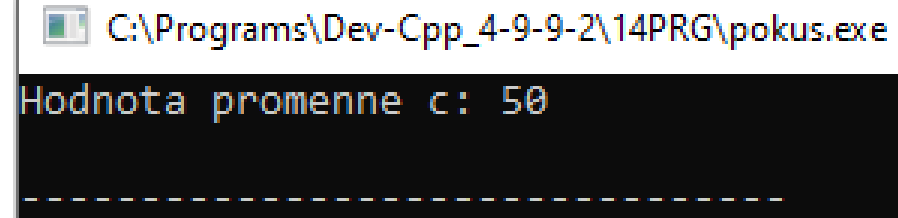
```
printf("Hodnota promenne c: %f",c);
```

```
printf("Hodnota promenne f: %d",f);
```

Výpis s odřádkováním

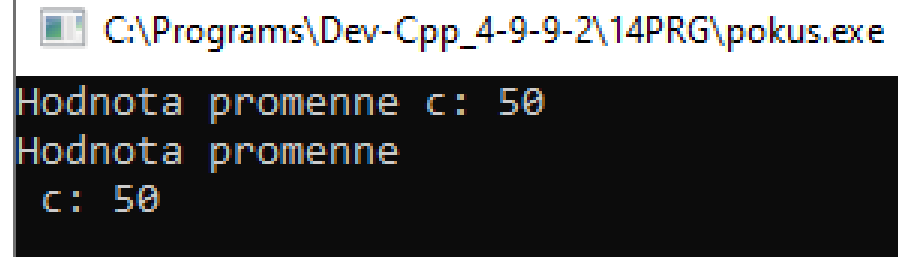
- Ve formátovacím řetězci pro odřádkování uvést `\n`

```
printf("Hodnota promenne c: %d\n",c);
```



```
C:\Programs\Dev-Cpp_4-9-9-2\14PRG\pokus.exe  
Hodnota promenne c: 50  
-----
```

```
printf("Hodnota promenne\n c: %d\n",c);
```



```
C:\Programs\Dev-Cpp_4-9-9-2\14PRG\pokus.exe  
Hodnota promenne c: 50  
Hodnota promenne  
c: 50
```

Zástupné znaky

zástupný znak	význam
\n	nový řádek
\t	tabulátor
\\	zpětné lomítko
\'	apostrof
\	řetězec

Výpis více hodnot

- Místo specifikátoru %... se vypisuje hodnota

```
printf("Hodnota a: %d \nhodnota c: %d\nsoucet %d\n",  
a,c,a+c);
```

 C:\Programs\Dev-Cpp_4-9-9-2\14PRG\pokus.exe

```
Hodnota a: 1  
hodnota c: 50  
soucet 51
```

Formátování výstupu

- viz <https://www.sallyx.org/sally/c/c07.php#printf>

```
float cislo=25.2264;  
printf("Hodnota promenne: %.2f",cislo);
```

Výstup textu na obrazovku

- pokud chceme pouze vypsát nějaký řetězec včetně odřádkování:

```
puts("Vypis a odradkovani");
```


Vstup z klávesnice

`scanf(...)` //nutná direktiva `#include <stdio.h>`

`scanf("%d", &c);`

Opravdu jenom VSTUP, nesnažte se použít pro výpis!!!

Specifikum Visual Studio

- funkce `scanf` je považována za nebezpečnou (vysvětlení viz zdroj níže), nutno použít makro

`_CRT_SECURE_NO_WARNINGS`

musí být uvedené před `#include <stdio.h>`

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <stdio.h>
```

Vstup z klávesnice

`scanf(...)` //nutná direktiva `#include <stdio.h>`

Je možné jedním příkazem načíst i více hodnot, ale je nutné si dát pozor při zadávání.

```
scanf("%d-%d",&c,&d);
```

formát zadávání řetězce z klávesnice:

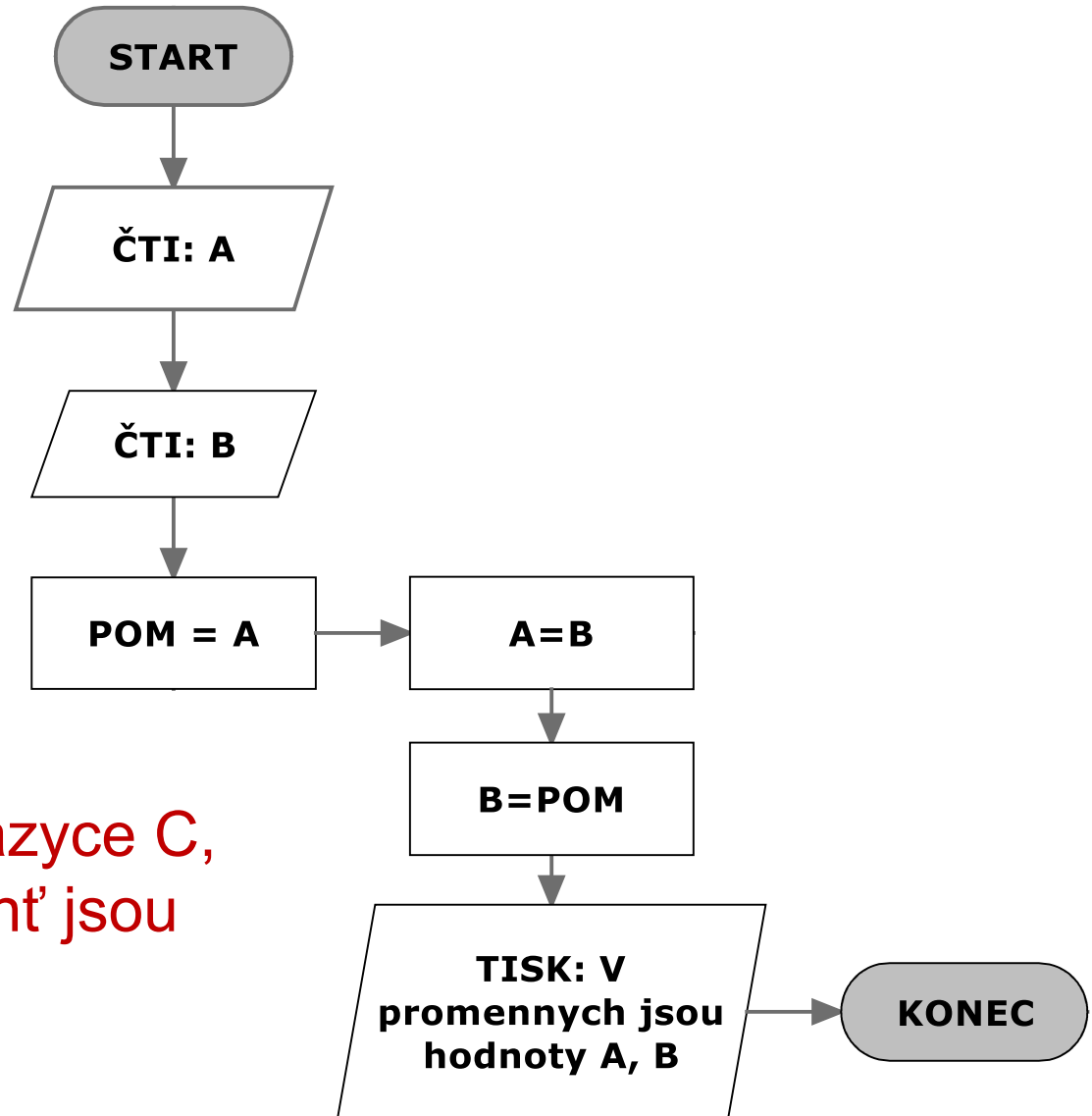
*číslo*mínus*číslo*enter, např. 34-21enter

Častá chyba

- Neuvedení &
- Špatně zvolený specifikátor

```
float c;  
scanf("%d",c);
```

Algoritmus výměny hodnot dvou proměnných



vytvořte program v jazyce C,
proměnné A a B necht' jsou
datového typu „int“

Algoritmus výměny hodnot dvou proměnných

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b, pom;
```

```
    printf("Zadej cislo A: ");
```

```
    scanf("%d",&a);
```

```
    printf("Zadej cislo B: ");
```

```
    scanf("%d",&b);
```

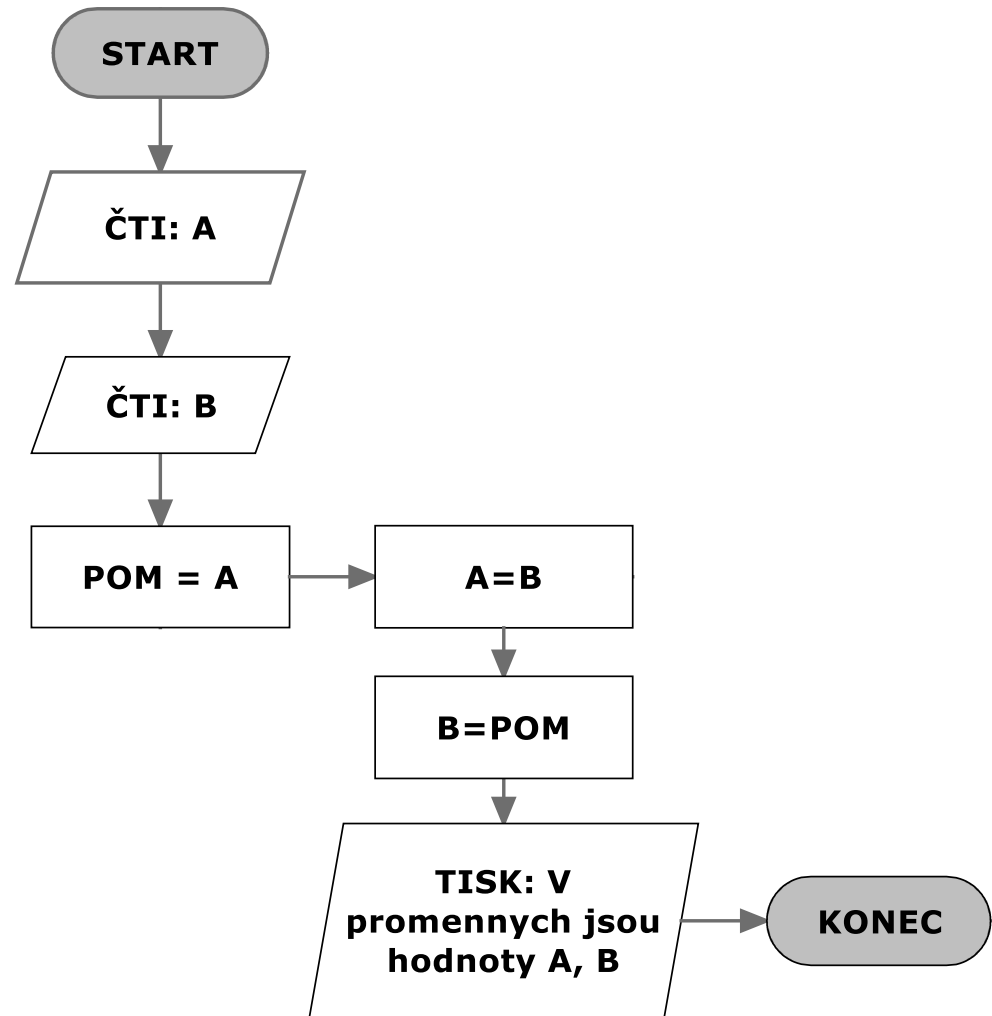
```
    pom=a;
```

```
    a=b;
```

```
    b=pom;
```

```
    printf("V promennych jsou hodnoty %d, %d",a,b);
```

```
}
```



CO JE VE ZDROJOVÉM
KÓDU A NENÍ VE
VÝVOJOVÉM DIAGRAMU?

KOMENTÁŘE

Komentář

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b, pom;
```

```
    // jednořádkový komentář printf("Zadej cislo A: ");
```

```
    scanf("%d",&a);
```

```
    /*víceřádkový komentář
```

```
        printf("Zadej cislo B: ");
```

```
        scanf("%d",&b);
```

```
    */
```

```
}
```

DEKLARACE KONSTANTY

Symbolická konstanta pomocí direktivy

```
#include <stdio.h>
```

```
#define PI 3.1415927
```

```
int main()
```

```
{
```

```
    double x;
```

```
    ...
```

```
    x = 2*PI;
```

```
    ...
```

```
}
```

Typová konstanta

```
#include <stdio.h>
```

```
const double PI = 3.1415927;
```

```
int main()
```

```
{
```

```
    double x;
```

```
    ...
```

```
    x = 2*PI;
```

```
    ...
```

```
}
```